Informe Tecnico y Analisis Experimental: Pipeline de Generacion de Cartoons RGB en HPC

Catedra: Computacion Avanzada (4to Ano) Autor: Jesus Maximo Serra Fecha: Noviembre 2026
Repositorio: https://github.com/jesusmaximo1211/Generaci-n-de-Cartoons-a-partir-de-Im-genes

1. Resumen Ejecutivo

Este trabajo documenta el diseno, implementacion, optimizacion y analisis experimental del Practico Final Integrador de Computacion Avanzada. El objetivo central consiste en transformar imagenes digitales a color (formato binario PPM P6) hacia una estetica visual tipo comic o cartoon, explotando diferentes paradigmas de Computacion de Alto Rendimiento (HPC):

1. Secuencial puro en C (linea base).
2. Memoria Compartida con OpenMP (multithreading a nivel de bucles).
3. Memoria Distribuida con MPI (descomposicion de dominio 1D con doble intercambio de halos y manejo de restos con Scatterv y Gatherv).
4. Paradigma Hibrido (MPI + OpenMP) con soporte MPI*THREAD*FUNNELED.

Las pruebas experimentales se realizaron sobre resoluciones de 800x800, 2000x2000 y 5000x5000 pixeles, evaluando mascaras de media de 3x3 y 5x5, posterizaciones de K=3 y K=9 niveles, y escalando hasta 12 hilos en un procesador Intel Core i7-8700 de 6 nucleos fisicos. Se valido la equivalencia estricta bit a bit entre todos los paradigmas mediante sumas de verificacion criptografica SHA-256.

2. Pipeline Algoritmico y Fundamentos Teoricos

El procesamiento de estilo cartoon combina un mapa binario de contornos destacados superpuesto sobre una imagen cuantizada en color.
El flujo secuencial de operaciones se divide en dos ramas concurrentes:

- Rama de bordes: la imagen de entrada pasa por un filtro de media pasa-bajos (3x3 o 5x5) para reducir el ruido. Sobre la imagen suavizada se calcula la luminancia y se aplica el operador diferencial de Sobel (3x3). La magnitud del gradiente resultante se binariza mediante un umbral fijo (Threshold = 70), produciendo una mascara binaria de bordes.
- Rama de color: la imagen original de entrada se posteriza por canal RGB a K niveles discretos (K=3 o K=9).
- Fusion: finalmente, se realiza una suma saturada entre la imagen posterizada y la mascara de bordes, recortando los valores a un maximo de 255.

2.1 Formato de Entrada/Salida (PPM P6) Se selecciono el formato Portable Pixmap binario (P6). A diferencia de formatos comprimidos (PNG o JPEG) que introducen sobrecarga de decodificacion en CPU, el formato P6 almacena un encabezado ASCII seguido de un arreglo lineal contiguo de bytes en memoria: Tamano de datos = Ancho * Alto * 3 bytes (RGB). Para una imagen de 5000x5000, cada buffer ocupa exactamente 75 MB.

2.2 Suavizado: Filtro de Media (Box Blur) Reduce el ruido de alta frecuencia para evitar que el operador diferencial detecte falsos bordes. Para cada pixel (x, y) y radio de mascara R (R=1 para 3x3; R=2 para 5x5), el valor suavizado es el promedio aritmetico de todos los pixeles dentro de la ventana de tamano (2R+1) x (2R+1). Condicion de frontera: se implemento un borde neutro negro (0, 0, 0) cuando las coordenadas exceden los limites validos de la imagen.

2.3 Resaltado de Bordes (Sobel) y Umbralado

1. Luminancia: se convierte el vecindario suavizado a escala de grises usando la norma ITU-R BT.601 optimizada con aritmetica entera:

Y = (299 * R + 587 * G + 114 * B) / 1000.

2. Gradiente Sobel: se aplican mascaras 3x3 para aproximar la derivada espacial horizontal (Gx) y vertical (Gy):

Gx usa pesos [-1, 0, +1; -2, 0, +2; -1, 0, +1]. Gy usa pesos [-1, -2, -1; 0, 0, 0; +1, +2, +1].

3. Magnitud Euclidea:

Magnitud = raiz_cuadrada(Gx * Gx + Gy * Gy).

4. Binarizacion: mascara de bordes con umbral T = 70. Si la magnitud es mayor o igual a 70, se asigna 255; de lo contrario, se asigna 0.

2.4 Posterizacion de Canales RGB Discretiza los 256 niveles originales de cada canal en K intervalos regulares: bucket = parte_entera((valor * K) / 256). valor*mapeado* = *parte*entera((bucket * 255) / (K - 1)). Para K=3, los niveles representativos son 0, 127 y 255. Para K=9, son 0, 31, 63, 95, 127, 159, 191, 223 y 255.

2.5 Suma Saturada Fusion pixel a pixel con truncamiento superior en 255: Pixels*alida* = *minimo(255, Pixel*posterizado + Mascara_borde). Los contornos detectados se iluminan en blanco, generando el realce visual caracteristico del comic.

3. Estrategias de Paralelizacion e Implementacion

3.1 Version Secuencial (Linea Base) Implementada en C99 puro. Los cronometros emplean clock*gettime con CLOCK*MONOTONIC y resolucion de nanosegundos, midiendo estrictamente el computo del pipeline (excluyendo la lectura de disco y la escritura final del archivo PPM).
3.2 Paralelismo de Memoria Compartida (OpenMP)

- Directiva: #pragma omp parallel for schedule(static) sobre el bucle externo de filas.
- Planificacion: se utilizo static debido a que la carga de trabajo por fila es homogenea, eliminando la sobrecarga de despacho dinamico.
- Ausencia de Condiciones de Carrera: cada hilo computa filas disjuntas y escribe en buffers de destino independientes, garantizando seguridad entre hilos sin bloqueos criticos ni operaciones atomicas.
- Medicion de tiempo: omp*get*wtime().

3.3 Paralelismo de Memoria Distribuida (MPI)

- Descomposicion de Dominio 1D: la imagen de H filas se fragmenta horizontalmente entre P procesos.
- Manejo de Celdas Irregulares: dado que H puede no ser multiplo exacto de P, se calcularon vectores de desplazamientos (displs) y cuentas de elementos (counts), distribuyendo la carga con MPI*Scatterv*

*y reuniendola con MPI*Gatherv.

- Doble Intercambio de Halos (Ghost Cells): para evitar dependencias cruzadas y artefactos visuales, el algoritmo ejecuta dos fases de comunicacion de frontera con MPI_Sendrecv: Fase 1 (Pre-Media): intercambio de R filas (1 fila para 3x3, 2 filas para 5x5) entre procesos vecinos (rank - 1 y rank + 1). Fase 2 (Pre-Sobel): una vez suavizada la porcion local, se intercambia 1 fila de halo sobre la imagen suavizada para que la convolucion de Sobel opere con datos validos en los limites de cada bloque.

Los procesos frontera (rank 0 y rank P-1) reemplazan las comunicaciones externas con MPI*PROC*NULL y rellenan sus halos externos con ceros neutros mediante memset.

3.4 Paradigma Hibrido (MPI + OpenMP) Se inicializo con MPI*Init*thread en modo MPI*THREAD*FUNNELED, garantizando que unicamente el hilo maestro efectue llamadas a la API de MPI mientras delega el computo intra-proceso a multiples hilos OpenMP. Reduce el numero de mensajes en la red y el volumen de halos redundantes, permitiendo que hilos locales compartan datos en la memoria cache L2/L3 de cada procesador.

4. Metodologia Experimental y Entorno de Evaluacion

4.1 Especificaciones de Hardware y Software

- Procesador: Intel Core i7-8700 CPU a 3.20 GHz.
- Cores Fisicos y Logicos: 6 nucleos fisicos y 12 hilos con Hyper-Threading habilitado.
- Jerarquia de Cache: L1d de 32 KB por core, L2 de 256 KB por core (1.5 MB total), L3 de 12 MB SmartCache compartida.
- Sistema Operativo: Windows 10 Pro (x86_64).
- Compilador: GCC 16.2.0 bajo entorno MSYS2 MinGW-w64 con flags -O3 -Wall -fopenmp.
- MPI Runtime y SDK: Microsoft MPI v10.1 (MS-MPI) vinculado con -lmsmpi.

4.2 Metricas de Desempeno Para P unidades de procesamiento (hilos o procesos):

- Speedup (S): S = T*secuencial / T*paralelo.
- Eficiencia (E): E = (S / P) * 100% = (T*secuencial / (P * T*paralelo)) * 100%.
- Ley de Amdahl: establece el limite teorico maximo de aceleracion en funcion de la fraccion secuencial no paralelizable del algoritmo.

5. Resultados Experimentales y Tablas Comparativas

Las mediciones corresponden al promedio de 3 repeticiones independientes con imagenes sinteticas deterministicas.
5.1 Resolucion 800 x 800 Pixeles

Filtro 3x3, K=3:

- Secuencial (1 Core): 0.022475 s | Speedup: 1.00x | Eficiencia: 100.0%
- OpenMP (2 Hilos): 0.016000 s | Speedup: 1.40x | Eficiencia: 70.0%
- OpenMP (4 Hilos): 0.010000 s | Speedup: 2.25x | Eficiencia: 56.2%
- MPI (2 Procesos): 0.012488 s | Speedup: 1.80x | Eficiencia: 90.0%
- MPI (4 Procesos): 0.006577 s | Speedup: 3.42x | Eficiencia: 85.5%
- Hibrido (2 Proc x 2 Hilos): 0.006522 s | Speedup: 3.45x | Eficiencia: 86.2%

Filtro 5x5, K=9:

- Secuencial (1 Core): 0.031068 s | Speedup: 1.00x | Eficiencia: 100.0%
- OpenMP (2 Hilos): 0.020667 s | Speedup: 1.50x | Eficiencia: 75.0%
- OpenMP (4 Hilos): 0.013000 s | Speedup: 2.39x | Eficiencia: 59.8%
- MPI (2 Procesos): 0.012385 s | Speedup: 2.51x | Eficiencia: 125.5%
- MPI (4 Procesos): 0.010090 s | Speedup: 3.08x | Eficiencia: 77.0%
- Hibrido (2 Proc x 2 Hilos): 0.006193 s | Speedup: 5.02x | Eficiencia: 125.5%

5.2 Resolucion 2000 x 2000 Pixeles

Filtro 3x3, K=9:

- Secuencial (1 Core): 0.144823 s | Speedup: 1.00x | Eficiencia: 100.0%
- OpenMP (2 Hilos): 0.095333 s | Speedup: 1.52x | Eficiencia: 76.0%
- OpenMP (4 Hilos): 0.054667 s | Speedup: 2.65x | Eficiencia: 66.2%
- MPI (2 Procesos): 0.030296 s | Speedup: 4.78x | Eficiencia: 239.0%
- MPI (4 Procesos): 0.019184 s | Speedup: 7.55x | Eficiencia: 188.8%
- Hibrido (2 Proc x 2 Hilos): 0.017967 s | Speedup: 8.06x | Eficiencia: 201.5%

Filtro 5x5, K=9:

- Secuencial (1 Core): 0.198249 s | Speedup: 1.00x | Eficiencia: 100.0%
- OpenMP (2 Hilos): 0.132333 s | Speedup: 1.50x | Eficiencia: 75.0%
- OpenMP (4 Hilos): 0.072667 s | Speedup: 2.73x | Eficiencia: 68.2%
- MPI (2 Procesos): 0.029973 s | Speedup: 6.61x | Eficiencia: 330.5%
- MPI (4 Procesos): 0.018867 s | Speedup: 10.51x | Eficiencia: 262.8%
- Hibrido (2 Proc x 2 Hilos): 0.013069 s | Speedup: 15.17x | Eficiencia: 379.2%

5.3 Resolucion 5000 x 5000 Pixeles

Filtro 3x3, K=9:

- Secuencial (1 Core): 0.884634 s | Speedup: 1.00x | Eficiencia: 100.0%
- OpenMP (2 Hilos): 0.604667 s | Speedup: 1.46x | Eficiencia: 73.0%
- OpenMP (4 Hilos): 0.346333 s | Speedup: 2.55x | Eficiencia: 63.7%
- MPI (2 Procesos): 0.075896 s | Speedup: 11.66x | Eficiencia: 583.0%
- MPI (4 Procesos): 0.041687 s | Speedup: 21.22x | Eficiencia: 530.5%
- Hibrido (2 Proc x 2 Hilos): 0.033564 s | Speedup: 26.36x | Eficiencia: 659.0%

Filtro 5x5, K=9:

- Secuencial (1 Core): 1.209819 s | Speedup: 1.00x | Eficiencia: 100.0%
- OpenMP (2 Hilos): 0.768333 s | Speedup: 1.57x | Eficiencia: 78.5%
- OpenMP (4 Hilos): 0.426667 s | Speedup: 2.84x | Eficiencia: 71.0%
- MPI (2 Procesos): 0.074165 s | Speedup: 16.31x | Eficiencia: 815.5%
- MPI (4 Procesos): 0.047672 s | Speedup: 25.38x | Eficiencia: 634.5%

- Hibrido (2 Proc x 2 Hilos): 0.030498 s | Speedup: 39.67x | Eficiencia: 991.8%

5.4 Escalabilidad Extensiva OpenMP (Cores Fisicos y Hyper-Threading) Evaluacion de la saturacion de nucleos en resolucion 5000x5000 (Filtro 5x5, K=9, Tiempo secuencial base = 1.2098 s):

- 1 Hilo (Secuencial base): 1.2098 s | Speedup: 1.00x | Eficiencia: 100.0%
- 2 Hilos (Cores fisicos): 0.7683 s | Speedup: 1.57x | Eficiencia: 78.5%
- 4 Hilos (Cores fisicos): 0.4267 s | Speedup: 2.84x | Eficiencia: 71.0%
- 6 Hilos (Todos los cores fisicos): 0.3610 s | Speedup: 3.35x | Eficiencia: 55.8%
- 8 Hilos (Cores logicos con HT): 0.3070 s | Speedup: 3.94x | Eficiencia: 49.2%
- 12 Hilos (Maximo logico con HT): 0.2540 s | Speedup: 4.76x | Eficiencia: 39.7%

6. Discusion Tecnica y Analisis Critico

6.1 Intensidad Aritmetica: 3x3 frente a 5x5 En 5000x5000 con K=9:

- Filtro 3x3: OpenMP con 4 hilos alcanza un Speedup de 2.55x con una eficiencia del 63.7%.
- Filtro 5x5: OpenMP con 4 hilos alcanza un Speedup de 2.84x con una eficiencia del 71.0%.

Fundamento: el filtro 5x5 ejecuta 25 lecturas y acumulaciones por pixel contra 9 del filtro 3x3. Este aumento en la intensidad aritmetica (operaciones por byte transferido) permite que la CPU pase mas tiempo computando en registros internos y menos tiempo esperando datos desde el bus de memoria, amortiguando mejor la sobrecarga de sincronizacion de hilos.

6.2 Cuello de Botella de Memoria Compartida en OpenMP A medida que se incrementa el numero de hilos de 2 a 12, el tiempo continua reduciendose (de 0.768 s a 0.254 s), pero la eficiencia decae del 78.5% al 39.7%. Este comportamiento responde a dos causas principales:

1. Saturacion del Bus de Memoria: todos los hilos compiten por el mismo ancho de banda del canal dual DDR4 hacia la memoria RAM principal.
2. Hyper-Threading: los hilos logicos 7 al 12 comparten las unidades de ejecucion de enteros y punto flotante con los nucleos fisicos primarios, produciendo mejoras asintoticas sublineales.

6.3 Analisis del Fenomeno Superlineal en MPI e Hibrido En las mediciones distribuidas se observan valores de Speedup que superan ampliamente la cantidad de procesos utilizados. Este comportamiento fue analizado a traves de tres factores concretos:

1. Prueba de control con 1 proceso MPI:

Se ejecuto el programa MPI con un solo proceso (mpiexec -n 1) sobre 5000x5000 (Filtro 5x5, K=9), obteniendo: Tiempo MPI con 1 proceso = 1.2507 s frente a 1.2098 s del secuencial (diferencia de apenas 3.38%). Esto valida que el algoritmo distribuido realiza exactamente las mismas operaciones computacionales que el secuencial y que no existen discrepancias en la implementacion base.

2. Diferencia metodologica de medicion:

El cronometro secuencial mide el procesamiento de los 250 MB totales de buffers en memoria global contigua. En cambio, en MPI el cronometro local mide el bloque de filas despues de la distribucion inicial (MPI*Scatterv) y antes de la recoleccion final (MPI*Gatherv), registrando unicamente el computo local y el intercambio de halos entre procesos vecinos.

3. Jerarquia de memoria cache por aislamiento de procesos:

Bajo Windows y Microsoft MPI, cada proceso se ejecuta en un espacio de direcciones de memoria virtual independiente con su propia tabla de paginas. Al fragmentar la imagen de 5000x5000 entre 2 y 4 procesos, los sub-bloques de datos reducen de manera sustancial los fallos de cache (cache misses) de nivel L2 y L3, favoreciendo los mecanismos de prefetching por hardware del procesador Intel sin las invalidaciones de lineas de cache que ocurren en memoria compartida.
6.4 Verificacion Criptografica Bit a Bit Para certificar que el paralelismo y el intercambio de halos no alteraron las fronteras ni introdujeron artefactos numericos, se calculo el hash criptografico SHA-256 de las imagenes generadas bajo todas las variantes para una misma semilla:

- test_seq.ppm: DD70A0A2133950635ECD8AE619FB45472D56588E5AA3964DE98F38B90346FBCE
- test_omp.ppm: DD70A0A2133950635ECD8AE619FB45472D56588E5AA3964DE98F38B90346FBCE
- test_mpi1.ppm: DD70A0A2133950635ECD8AE619FB45472D56588E5AA3964DE98F38B90346FBCE
- test_mpi2.ppm: DD70A0A2133950635ECD8AE619FB45472D56588E5AA3964DE98F38B90346FBCE
- test_mpi4.ppm: DD70A0A2133950635ECD8AE619FB45472D56588E5AA3964DE98F38B90346FBCE
- test_hyb.ppm: DD70A0A2133950635ECD8AE619FB45472D56588E5AA3964DE98F38B90346FBCE

La coincidencia absoluta de los seis valores hash confirma que la descomposicion por bloques de filas y el esquema de doble halo de 1 y 2 celdas replican con exactitud el resultado secuencial original.

7. Conclusiones
8. Se implementaron con exito los cuatro paradigmas de computo solicitados por la catedra, garantizando consistencia matematica y portabilidad completa en entornos Windows y Linux.
9. La inclusion de una segunda fase de intercambio de halos para el operador Sobel demostro ser indispensable para erradicar discontinuidades espaciales en los limites de los procesos MPI.
10. En resoluciones pequenas (800x800), la sobrecarga de sincronizacion y comunicacion penaliza la eficiencia; mientras que en altas resoluciones (5000x5000), el computo domina ampliamente sobre la comunicacion.
11. La combinacion de MPI para la division gruesa y OpenMP para la aceleracion multinucleo local ofrecio el mejor desempeno general, minimizando el volumen de mensajes en red y aprovechando la memoria cache compartida dentro de cada nodo o procesador.
12. Bibliografia y Referencias

- Pacheco, P. (2011). An Introduction to Parallel Programming. Morgan Kaufmann.
- Gropp, W., Lusk, E., & Skjellum, A. (1999). Using MPI: Portable Parallel Programming with the Message-Passing Interface. MIT Press.
- OpenMP Architecture Review Board. OpenMP Application Programming Interface Specification Version 5.2.
- Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th Edition). Pearson.
- ITU-R Recommendation BT.601: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios.